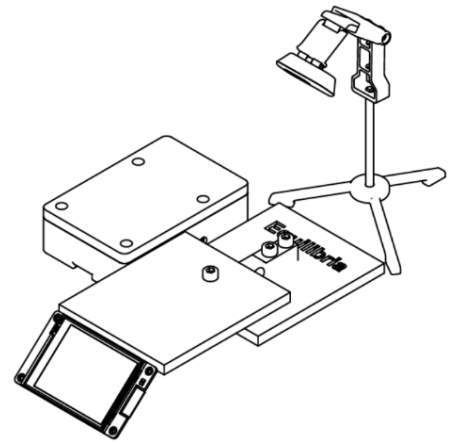


Equilibria

Balança comercial inteligente



Catarina Costa, Diogo Oliveira, Gabriel Silva e Inês Almeida

Departamento de Informática
Instituto Politécnico de Viseu
Viseu, Portugal
{pv27433, pv27451, pv30904,
pv25225}@alunos.estgv.ipv.pt

Resumo - Este artigo apresenta o desenvolvimento do sistema Equilibria, uma balança de pesagem inteligente concebida no âmbito da unidade curricular de Sistemas Embebidos. A solução proposta recorre a sistemas embebidos e visão computacional para a identificação automática de frutas, eliminando a necessidade de introdução manual de códigos.

Palavras Chave - sistemas embebidos; visão computacional; reconhecimento de frutas; balança inteligente.

I. INTRODUÇÃO

A pesagem de frutas em regime de self-service continua a ser uma tarefa suscetível a erros e atrasos nos supermercados, devido à necessidade de introdução manual de códigos de produto por parte dos utilizadores. Este processo não só compromete a experiência do cliente, como também pode originar perdas económicas para os estabelecimentos.

Neste contexto, o projeto Equilibria propõe uma balança inteligente baseada em sistemas embebidos e visão computacional, capaz de identificar automaticamente frutas sem recurso a códigos. A solução integra uma câmara e um modelo de Inteligência Artificial, permitindo a identificação do produto, bem como a medição do seu peso através de células de carga. Possui, igualmente, uma infraestrutura de software que assegura a gestão dos produtos e dos respetivos preços.

Desta forma, o Equilibria contribui para a automatização do processo de pesagem, melhorando a eficiência do serviço e a experiência do utilizador em ambientes de self-service.

Este artigo científico encontra-se organizado em cinco secções: “Estado da Arte”, “Arquitetura do Sistema”, “Modelo de Inteligência Artificial”, “Funcionamento do Sistema” e “Resultados e Melhorias Futuras”.

II. ESTADO DA ARTE

A pesagem de produtos com a introdução manual de códigos apresenta limitações, como erros de codificação, atrasos no atendimento e impacto negativo na experiência do cliente.

Atualmente, soluções mais recentes exploram visão computacional e inteligência artificial para identificar automaticamente produtos, nomeadamente frutas e legumes. Alguns projetos utilizam câmaras e modelos para deteção de objetos para classificar os produtos, mas muitas destas abordagens permanecem apenas em fase experimental e não estão implementadas em supermercados.

Deste modo, visualizamos a importância de desenvolver este tipo de sistema de pesagem inteligentes integrando hardware embebido, visão computacional e gestão dos produtos em ambientes de self-service, contribuindo para uma melhor experiência do utilizador.

III. ARQUITETURA DO SISTEMA

A arquitetura do sistema foi concebida de forma modular, dividindo-se em componentes de software e hardware, cuja interação é ilustrada no diagrama de blocos da [Figura 1](#).

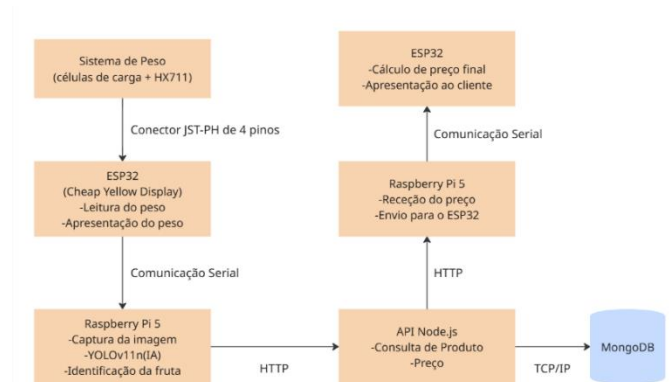


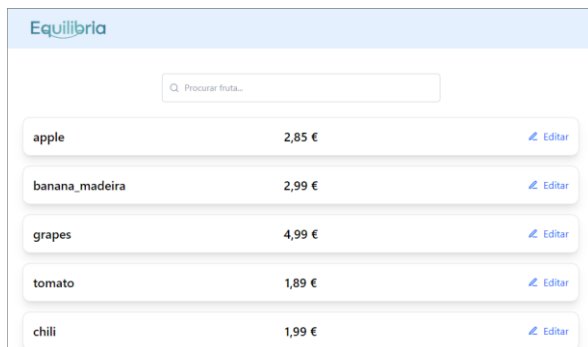
Figura 1 - Arquitetura do sistema

A. Arquitetura de software

A componente de *software* é sustentada por uma **API REST** desenvolvida em **Node.js**, que atua como intermediária entre os dispositivos físicos e os dados. Esta API é responsável pela lógica de negócio e pela consulta de informações detalhadas dos produtos (designação e preço por quilo).

A persistência de dados é assegurada por uma base de dados não relacional **MongoDB**, alojada na cloud (**MongoDB Atlas**), garantindo escalabilidade e acesso remoto.

Complementarmente, foi desenvolvida uma interface de gestão em **React**, permitindo aos administradores do sistema a atualização em tempo real dos respetivos preços dos produtos.



Equilibria		
Q. Procurar fruta...		
apple	2,85 €	✎ Editar
banana_madeira	2,99 €	✎ Editar
grapes	4,99 €	✎ Editar
tomato	1,89 €	✎ Editar
chili	1,99 €	✎ Editar

Figura 2 - Interface de gestão

B. Arquitetura de Hardware

A arquitetura de hardware está dividida em dois módulos funcionais: o **módulo de pesagem** e o **módulo de identificação do produto**.

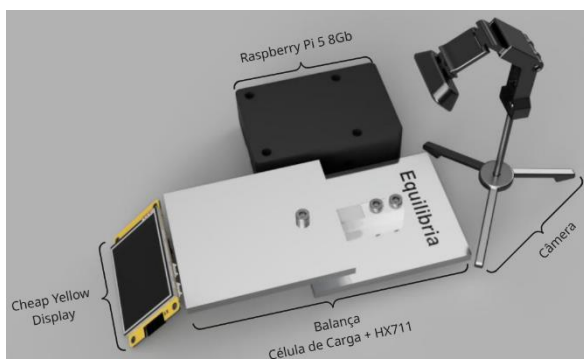


Figura 3 - Vista geral da arquitetura de hardware e integração dos componentes

O **módulo de pesagem** é constituído por um **microcontrolador ESP32**, integrado no **Cheap Yellow Display**, responsável pela leitura do peso através das **células de carga** ligadas ao conversor **HX711**. Neste módulo foi desenvolvida uma interface gráfica recorrendo à biblioteca **LVGL**, que permite ao utilizador visualizar, em tempo real, o peso medido pela balança.

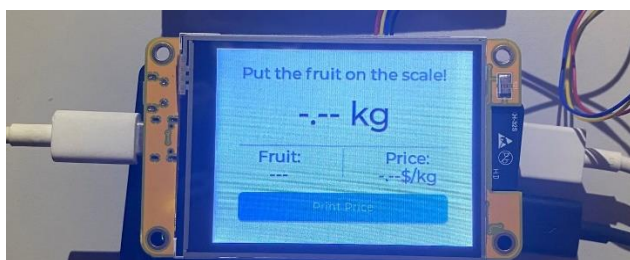


Figura 4 - Interface gráfica do CYD

O módulo de identificação do produto é baseado num **Raspberry Pi 5** (8 GB RAM), responsável pela execução do modelo de **Inteligência Artificial YOLO** para a identificação automática de frutas e legumes. O processo de identificação é iniciado apenas quando o ESP32 deteta a presença de peso na balança, enviando essa informação ao Raspberry Pi através de **comunicação serial**.

Após a identificação do produto, o **Raspberry Pi** comunica com a **API** através do protocolo **HTTP**, de forma a obter o preço correspondente ao produto identificado. A informação recebida é posteriormente enviada de volta ao ESP32 via comunicação serial, permitindo apresentar no visor o produto identificado e o respetivo preço por quilograma.

Uma vez concluídas a identificação e a estabilização da pesagem, o sistema habilita um botão na interface. Ao acionar esta opção, o utilizador obtém o cálculo do valor final a pagar no ecrã.



Figura 5 - Protótipo final do sistema Equilibria, demonstrando a identificação e pesagem de uma maçã

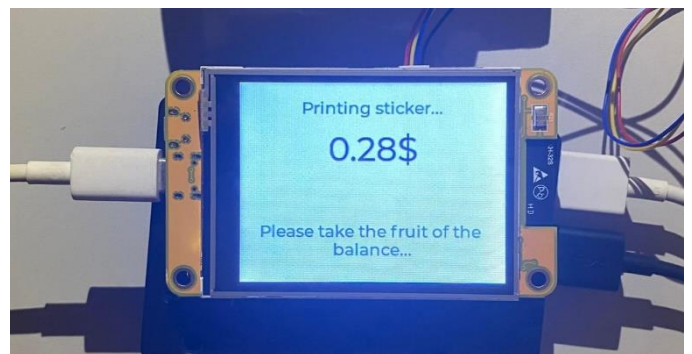


Figura 6 - Ecrã de confirmação com o valor a pagar após acionamento do botão

IV. MODELO DE INTELIGÊNCIA ARTIFICIAL

A. Seleção do modelo

Para a tarefa de deteção de objetos, optou-se pela utilização da arquitetura **YOLO** (You Only Look Once), reconhecida pela sua eficiência em tempo real. A escolha da versão específica foi condicionada pelas restrições de hardware do **Raspberry Pi 5** (8 GB RAM), devido à sua limitada capacidade de processamento gráfico (GPU) para executar redes neurais localmente.

Foi realizada uma pesquisa para comparar as variantes "nano", especificamente YOLOv5n, YOLOv8n e YOLOv11n. A análise técnica concluiu que o YOLOv11n oferece o melhor compromisso (trade-off) entre precisão e latência

computacional, apresentando melhorias significativas, o que o torna a solução ideal para o sistema proposto.

B. Metodologia de treino

O **data-set público**^[2] utilizado provém da plataforma **Kaggle**, composto por diversas classes de vegetais e frutas. O processo de anotação nas imagens foi realizado através das ferramentas **Label Studio** e **CVAT**, garantindo a qualidade e consistência das labels, totalizando **534 imagens anotadas** referentes a **8 classes de frutas e vegetais**.

O treino do modelo foi executado em ambiente cloud utilizando o **Google Colab**^[3], tirando partido da aceleração por hardware (GPU - Tesla T4) disponível na plataforma. O processo envolveu múltiplos ciclos de treino para maximizar a capacidade de detecção do modelo final, tendo este sido submetido a um ciclo de treino de **100 épocas**.

C. Análise de desempenho

A avaliação do modelo YOLOv11n foi realizada com base nas métricas padrão de detecção de objetos, considerando um treino ao longo de 100 épocas.

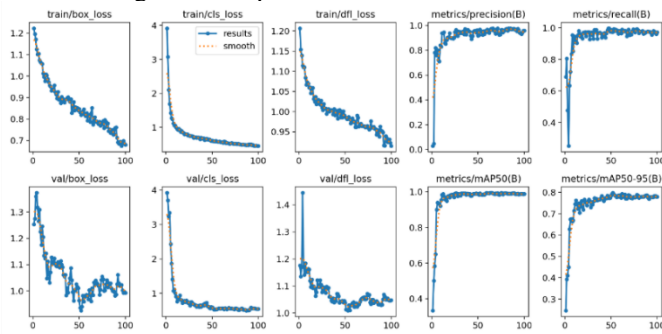


Figura 7 - Resultados dos testes

A **Figura 7** apresenta a evolução das métricas de desempenho e das funções de perda (loss functions) durante o processo de treino. Observa-se uma convergência rápida e estável, onde as curvas de box_loss (erro na localização das caixas delimitadoras) e cls_loss (erro na classificação das frutas) apresentam um decréscimo acentuado nas primeiras 20/30 épocas, estabilizando progressivamente até à última época.

Simultaneamente, as métricas de precisão (Precision) e Recall mostram um crescimento consistente, indicando que o modelo não sofreu de **overfitting** significativo e foi capaz de generalizar as características visuais das frutas presentes no dataset.

Overfitting – quando o modelo aprende demais os dados do treino, levando-o a memorizar padrões isto leva à perda na sua capacidade de detecção.

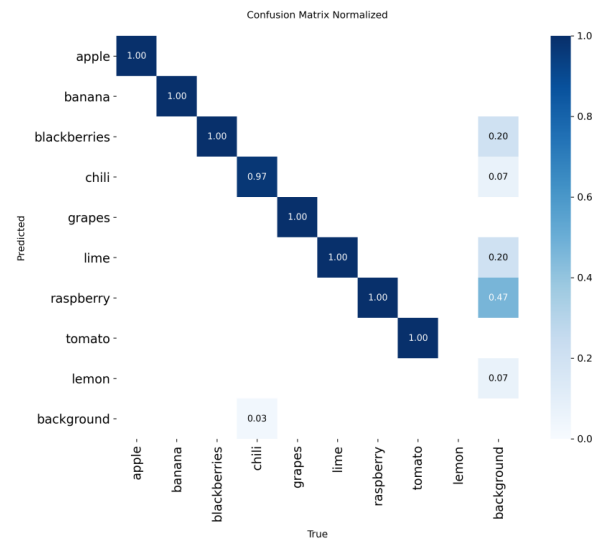


Figura 8 - Matriz de confusão normalizada

O modelo alcançou uma Mean Average Precision (mAP@0.5) global de 0.991, podemos visualizar isso ainda na **Figura 7**, o que demonstra uma excelente capacidade de distinção entre as classes.

No entanto, uma análise mais detalhada através da Matriz de Confusão Normalizada, apresentada na **Figura 8**, revela algumas dificuldades no desempenho entre diferentes tipos de fruta.

Verifica-se que frutas com características visuais distintas e de maior dimensão, como a maçã (*apple*), a banana (banana) e o tomate (tomato), obtiveram uma taxa de acerto muito boa (1.00). Em contrapartida, as classes correspondentes a frutos de menores dimensões apresentaram taxas menores, é o exemplo da framboesa (raspberry) e da amora (blackberry).

Estas falhas podem ser atribuídas a dois fatores principais: a reduzida dimensão destes frutos, que dificulta a extração das suas características, e o desequilíbrio do conjunto de dados (*dataset imbalance*), onde detetamos que certas classes possuíam significativamente menos amostras para treino do que outras.

Além disso, após pesquisa técnica detetamos a vantagem da conversão do modelo YOLO para o formato **NCNN**. Esta *framework* é especificamente otimizada para arquiteturas **ARM64**, como a utilizada pelo processador do Raspberry Pi 5.

Procedeu-se, assim, à conversão do modelo, resultando numa melhoria significativa do desempenho do sistema.

Por fim, o sistema apresentou uma elevada fiabilidade na maioria dos produtos testados, embora careça ainda de otimização em casos específicos. Prevê-se que a expansão do conjunto de dados (*dataset*) e a realização de mais testes complementares permitiriam melhorar o desempenho do

modelo, validando a sua robustez para uma futura implementação em ambiente de supermercado.

V. FUNCIONAMENTO DO SISTEMA

O sistema Equilibria destaca-se pela elevada facilidade de utilização (user-friendly). O utilizador necessita apenas de colocar o produto na balança, aguardar a identificação automática e confirmar a operação através da interface gráfica para a impressão do talão. Este modelo de interação dispensa qualquer adaptação comportamental significativa, agilizando o processo de checkout em comparação com os métodos tradicionais.

VI. RESULTADOS E MELHORIAS FUTURAS

Ao longo do desenvolvimento do projeto Equilibria, os resultados obtidos demonstraram-se bastante promissores, destacando-se a eficácia na integração entre a deteção visual e o sistema de pesagem. Embora a fase inicial de implementação tenha apresentado desafios técnicos em ambos os módulos, a abordagem iterativa de testes e correções permitiu refinar o protótipo final, alcançando um nível de estabilidade satisfatório.

No que toca ao modelo de IA, reconhece-se que existe margem para otimização. Sendo este o primeiro contacto da equipa com redes neurais de deteção de objetos, e considerando as restrições de tempo do projeto, o modelo atual apresenta limitações no número de classes e na precisão do mesmo. Contudo, os resultados alcançados conseguem mostrar o conceito deste mesmo projeto.

Adicionalmente, ao nível da experiência do utilizador (UX), propõe-se a reformulação da interface gráfica. O objetivo será desenvolver um sistema visual mais minimalista e heurístico, garantindo uma acessibilidade e uma interação intuitiva para qualquer utilizador.



Figura 9 - Conceito do futuro produto Equilibria integrado num ambiente real de retalho

Como trabalho futuro, pretende-se expandir o dataset e realizar um treino mais exaustivo para aproximar o desempenho

do sistema às exigências de um cenário real de retalho (real-world scenario), como mostrado na [Figura 9](#).

VII. CONCLUSÃO

O desenvolvimento e a implementação do sistema Equilibria demonstraram, de forma consistente, a viabilidade técnica de uma solução de balança inteligente baseada em sistemas embebidos e visão computacional para a identificação e pesagem automática de frutas em ambientes de self-service. A arquitetura proposta, assente na integração entre um microcontrolador ESP32 e um Raspberry Pi 5, permitiu uma comunicação eficiente e assíncrona entre os módulos de pesagem e de identificação do produto, garantindo um funcionamento estável e responsivo do sistema.

A adoção do modelo de deteção de objetos YOLOv11n revelou-se adequada às restrições de hardware impostas, apresentando um equilíbrio eficaz entre precisão e latência computacional. Os resultados obtidos evidenciam uma elevada capacidade de deteção para a maioria das classes testadas, validando a abordagem adotada. No entanto, foram identificadas limitações ao nível da discriminação de frutos de pequenas dimensões e em classes com menor representatividade no conjunto de dados, refletindo o impacto do *dataset imbalance* no desempenho do modelo.

Apesar destas limitações, o sistema mostrou-se particularmente relevante do ponto de vista experimental e aplicacional, comprovando o potencial da utilização de técnicas de Inteligência Artificial em soluções reais de automação no retalho. O protótipo desenvolvido constitui uma prova de conceito funcional, capaz de melhorar a eficiência do processo de pesagem e de reduzir erros associados à introdução manual de códigos.

Em termos globais, o projeto Equilibria apresenta um elevado potencial de evolução futura. A expansão do conjunto de dados, aliadas a um treino mais exaustivo do modelo e a melhorias na interface gráfica, poderão aproximar o sistema das exigências de um cenário real de supermercado. Desta forma, o Equilibria poderá servir como base para o desenvolvimento de uma solução comercial robusta no domínio da distribuição alimentar automatizada.

REFERÊNCIAS BIBLIOGRÁFICA

1. Ultralytics, "[YOLOv11 Train Mode Documentation](#)".
2. K. Patel, "[Fruits & Vegetable Detection for YOLOv4](#)".
3. Edge Electronics, "[How to Train YOLO Object Detection Models in Google Colab \(YOLO11, YOLOv8, YOLOv5\)](#)".
4. RNT "[LVGL with ESP32 Cheap Yellow Display Board \(ESP32-2432S028R\)](#)".
5. RPI "[Getting started – Raspberry Pi](#)".
6. Edge Electronics, "[How to Run YOLO Object Detection Models on the Raspberry Pi](#)".